

**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное бюджетное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана**
**(национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

# ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1
# ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

***Обработка больших чисел***

**Вариант № 3**

Студент: **Чернышева А. П.**

Группа: **ИУ7-36Б**

Преподаватель: **Барышникова М. Ю.**

**2025 г.**

## Описание условия задачи

Смоделировать операцию умножения целого числа длиной до 40 десятичных цифр на действительное число в форме ±m.n E ±K, где суммарная длина мантиссы (m+n) - до 30 значащих цифр, а величина порядка K - до 5 цифр. Результат выдать в форме ±0.m1 E ±K1, где m1 - до 40 значащих цифр, а K1 - до 5 цифр.

## Описание ТЗ

### 1. Описание исходных данных

Программа получает данные через стандартный ввод с помощью функции fgets, которая считывает данные в строки (тип char). Ввод состоит из двух строк:

**Строка 1: Целое число**

Формат: [+/-]X

1. [+/-] — необязательный знак числа. Если знак отсутствует, число считается положительным.
2. X — последовательность цифр (целое число).

Ограничение: Количество цифр в числе X не должно превышать 40.

**Строка 2: Действительное число в экспоненциальной форме**

Формат: [+/-]X(m[.n]) E [+/-]K

1. [+/-] перед X — необязательный знак мантиссы.
2. X — последовательность цифр мантиссы.
3. E — символ экспоненты (буква 'E' в верхнем или нижнем регистре).
4. [+/-]K — обязательный знак и цифры порядка.

Ограничения:

Суммарное количество значащих цифр в мантиссе X — до 30.

Длина порядка K — до 5 цифр.

Общие ограничения:

- Длина порядка не должна превышать 5 цифр,
- Порядок имеет до 5 знаков: от –99999 до +99999.

### 2. Описание результатов

Результат операции умножения должен быть выведен в нормализованной экспоненциальной форме:

Формат: +/-0.m1 E ±K1

1. +/- — знак результата.
2. 0.m1 — мантисса, представленная в виде дробной части с ведущим нулём.
3. m1 — последовательность значащих цифр.
4. E — символ экспоненты.
5. +/-K1 — знак и значение порядка.

Ограничения:

1. Длина мантиссы m1 — до 40 значащих цифр.
2. Длина порядка K1 — до 5 цифр.

### 3. Описание задачи, реализуемой в программе

Цель работы

Смоделировать операцию умножения двух больших чисел:

1. Целое число длиной до 40 десятичных цифр.
2. Действительное число в экспоненциальной форме записи.

Результат умножения должен быть представлен в нормализованной экспоненциальной форме.

**4. Способ обращения к программе**

Через исполняемый файл(app.exe).

**5. Описание возможных аварийных ситуаций и ошибок пользователя**

1. Наличие недопустимых символов (букв, специальных символов кроме '+', '-', '.', 'E') в числах
2. Наличие нескольких точек в действительном числе
3. Длина мантиссы целого числа превышает 40 цифр
4. Длина мантиссы действительного числа превышает 30 цифр
5. Количество цифр в порядке числа превышает 5 символов
6. Значение результирующей экспоненты превышает допустимый диапазон (+/-99999)
7. Использование символов, отличных от '+' и '-' для указания знака
8. Отсутствие знака при обязательном формате (для порядка числа)
9. Превышение максимальной длины входных строк (более 50 символов)
10. Ошибки ввода/вывода при чтении данных
11. Пустые входные строки
12. Ошибки операционной системы

## Описание внутренних структур данных

Программа содержит две структуры. Одна для целого числа и результата умножени, а вторая для вещественного числа.

Листинг структуры для целого числа и результата умножения

```
struct long_num {
    char sign;
    char mantissa[MAX_INT_MANT + 1];
    int exponent;
    int len_m;
};
```

- sign (char) - знак числа: '+' (положительное) или '-' (отрицательное)
- mantissa[] (char array) - массив символов, представляющий мантиссу числа - без десятичной точки
- exponent (int) - порядок числа
- len_m (int) - длина мантиссы

Листинг структуры для целого числа и результата умножения

```
struct short_num {
    char sign;
    char mantissa[MAX_LEN_MANT + 1];
    int exponent;
    int len_m;
};
```

- sign (char) - знак числа: '+' (положительное) или '-' (отрицательное)

- mantissa[] (char array) - массив символов, представляющий мантиссу числа - без десятичной точки
- exponent (int) - порядок числа
- len_m (int) - длина мантиссы

**Описание алгоритма**

1. Ввод и обработка входных данных
2. Проверка корректности ввода
3. Выполнение операции умножения
4. Нормализация результата
5. Вывод результата в требуемом формате

Описание функций:

**Функция parse_integer (const char* str, struct long_num* result):**

1. Принимает строку и указатель на структуру long_num.
2. Устанавливает начальные значения знака, мантиссы, экспоненты и длины.
3. Обрабатывает знак числа (+ или -).
4. Пропускает ведущие нули в строке.
5. Если строка состоит только из нулей, устанавливает мантиссу в "0".
6. Записывает цифры в мантиссу числа, проверяя на недопустимые символы.
7. Устанавливает длину мантиссы и завершает строку нулевым символом.
8. Возвращает ERROR при ошибках или 0 при успешном выполнении.

**Функция parse_float (const char* str, struct short_num* result):**

1. Принимает строку и указатель на структуру short_num.
2. Устанавливает начальные значения знака, мантиссы, экспоненты и длины.
3. Обрабатывает знак числа (+ или -).
4. Пропускает ведущие нули до десятичной точки.
5. Обрабатывает мантиссу числа, учитывая десятичную точку.
6. Подсчитывает общее количество цифр и проверяет ограничение MAX_LEN_MANT.
7. Удаляет конечные нули из мантиссы и корректирует экспоненту.
8. Обрабатывает экспоненту числа, если присутствует.
9. Проверяет допустимость символов и ограничение MAX_EXPONENT.
10. Возвращает ERROR при ошибках или 0 при успешном выполнении.

**Функция multiply_numbers(const struct long_num* a, const struct short_num* b, struct long_num* result):**

1. Принимает два числа (long_num и short_num) и указатель на результат.
2. Определяет знак результата на основе знаков входных чисел.
3. Вычисляет экспоненту результата как сумму экспонент.
4. Преобразует мантиссы в массивы цифр.
5. Выполняет умножение в столбик, сохраняя промежуточные результаты.
6. Обрабатывает переносы при умножении.
7. Пропускает ведущие нули в результате.
8. Если результат равен нулю, устанавливает соответствующие значения.
9. Записывает результат в мантиссу и устанавливает длину.
10. Корректирует экспоненту результата.

11.Возвращает 0 при успешном выполнении.

**Функция print_normalized(const struct long_num* num):**

1. Принимает указатель на структуру long_num.
2. Проверяет, является ли мантисса пустой, и выводит "+0.0 E +0" в этом случае.
3. Проверяет допустимость значения экспоненты.
4. Выводит знак числа в формате "+" или "-".
5. Выводит мантиссу в формате "0.123456".
6. Выводит экспоненту в формате "E +123".
7. Возвращает ERROR при ошибках или 0 при успешном выполнении.

## Набор тестов

### Позитивные тесты

| | Вход | Выход |
|---|---|---|
| Обычное умножение | 123<br>4.5 E -1 | +0.5535 E +2 |
| Одно отрицательное число(целое) | -123<br>4.5 E 0 | -0.5535 E +3 |
| Оба отрицательных числа | -123<br>-1.5 E +1 | +0.1845 E +4 |
| Одно отрицательное число(вещественное) | 123<br>-1.5 E +1 | -0.1845 E +4 |
| В целом числе есть незначащие нули | 000123<br>-1.5 E +1 | -0.1845 E +4 |
| В вещественном числе есть незначащие нули | 123<br>000.45600 E 0 | +0.56088 E +2 |
| Максимальной длины целое число | 99999999999999999999999999999999999999<br>1.0 E 0 | +0.99999999999999999999999999999999999999 E +40 |
| Максимальной длины | 2 | +0.19999999999999999999999999999998 E +31 |

| | | |
|---|---|---|
| мантисса вещественного числа с округлением | 99999999999999999999999999999 E 0 | |
| Максимальной длины экспонента вещественного числа | 2<br>0.02 E 99999 | +0.4 E +99998 |
| Умножение на ноль | 12<br>0 | +0.0 E +0 |

**Негативные тесты**

| | Вход | Выход |
|---|---|---|
| Пустая строка целого числа | <br>1 E-6 | Ошибка ввода |
| Пустая строка действительного числа | 123 | Ошибка ввода |
| В строке целого содержатся нечисловые символы | 1t23wre<br>1 E-6 | Ошибка ввода |
| В строке действительного числа содержатся нечисловые символы | 1<br>1 EE -6 | Ошибка ввода |
| Переполнение порядка | 1<br>9999 E 99999 | Переполнение порядка |
| Переполнение длины мантиссы | 1<br>11111111111111111111111111111111 E -6 | Ошибка ввода |
| Переполнение длины целого числа | 11111111111111111111111111111111111111 | Ошибка ввода |
| Вещественное число состоит только из пробелов | 1 | Ошибка ввода |

## Вывод

Для того чтобы выполнить умножение чисел, превосходящих разрядную сетку можно это делать по-разрядно с помощью алгоритма умножения мантисс в столбик и сложения порядков экспонент. Реализация алгоритма потребовала создания структуры данных, предназначенной для работы с длинными числами.

## Контрольные вопросы

**Каков возможный диапазон чисел, представляемых в ПК?**

Для 16-разрядных целых чисел: до 65 535 ($2^{16}$-1)

Для 32-разрядных целых чисел: до 4 294 967 295 ($2^{32}$-1)

Для 64-разрядных целых чисел: до 18 446 744 073 709 551 615 ($2^{64}$-1)

Для вещественных чисел: от $3.6\times10^{-4951}$ до $1.1\times10^{+4932}$ (мантисса – 52 двоичных разряда, порядок - 11 разрядов)

**Какова возможная точность представления чисел, чем она определяется?**

Точность целых чисел: определяется количеством разрядов, выделенных под представление числа

Точность вещественных чисел: определяется длиной мантиссы (до 52 двоичных разрядов)

Для высокоточных вычислений (20-30 знаков после точки) требуется реализация программистом

**Какие стандартные операции возможны над числами?**

Сложение, вычитание, умножение, деление, сравнение.

**Какой тип данных может выбрать программист, если обрабатываемые числа превышают возможный диапазон представления чисел в ПК?**

Массив символов (каждая цифра - отдельный элемент)

Массив чисел (хранение групп цифр)

Структуры с полями для знака, мантиссы и порядка

Представление в системе счисления с большим основанием (например, 10000)

**Как можно осуществить операции над числами, выходящими за рамки машинного представления?**

Использование алгоритмов "в столбик" для умножения и деления

Хранение чисел в перевернутом виде (младшие разряды сначала)

Обработка по тетрадам (группам по 4 цифры)

Ручное управление переносами между разрядами

Нормализация чисел перед выполнением операций

Реализация округления при превышении разрядности мантиссы